

CSE422: Artificial Intelligence

Lab Project Report

Stroke Prediction Using Machine Learning

Name: Navid Nawaj

Student ID: 22101838

Name: Muhammad Ashfaq

Tahmeed

Student ID: 22101211

Section: 02

Date: May 05, 2026

Table of Contents

1. Introduction
2. Dataset Description
 - 2.1 Overview
 - 2.2 Correlation Analysis
 - 2.3 Imbalanced Dataset
 - 2.4 Exploratory Data Analysis
3. Dataset Pre-processing
 - 3.1 Irrelevant Feature
 - 3.2 Missing Values
 - 3.3 Rare Category
 - 3.4 Categorical Encoding
 - 3.5 Feature Scaling
4. Dataset Splitting
5. Model Training & Testing
 - 5.1 Neural Network
 - 5.2 Logistic Regression
 - 5.3 Decision Tree
 - 5.4 KMeans Clustering (Unsupervised)
6. Model Selection / Comparison Analysis
7. Conclusion

1. Introduction

Stroke is one of the leading causes of death and disability worldwide. According to the World Health Organization, stroke is the 2nd leading cause of death globally, responsible for approximately 11% of total deaths. Early prediction of stroke risk based on patient health data can help in timely medical intervention and potentially save lives.

This project aims to build a complete Machine Learning pipeline to predict whether a patient is likely to have a stroke based on various health and demographic features. We use the **Healthcare Dataset – Stroke Data**, which contains 5,110 patient records with 12 attributes including age, gender, BMI, glucose level, and other clinical indicators.

The target variable `stroke` is binary (1 = stroke, 0 = no stroke), making this a **binary classification** problem. We train three supervised models (Neural Network, Logistic Regression, Decision Tree) and one unsupervised model (KMeans), then compare their performance using standard classification metrics.

2. Dataset Description

2.1 Overview

Property	Details
Number of data points	5,110
Number of columns	12
Problem type	Classification — the target <code>stroke</code> is binary (0 or 1)
Quantitative features	age, avg_glucose_level, bmi
Categorical features	gender, ever_married, work_type, Residence_type, smoking_status
Binary numeric features	hypertension, heart_disease
Target variable	stroke (0 = No Stroke, 1 = Stroke)
Identifier (dropped)	id

Why is this a classification problem? The target variable `stroke` has discrete class labels (0 or 1) rather than continuous values. We are predicting which class a patient belongs to, which is the definition of classification.

Do we need to encode categorical variables? Yes. Machine learning models require numerical inputs. Categorical features such as `gender`, `work_type`, `Residence_type`, `smoking_status`, and `ever_married` must be converted to numerical form using encoding techniques.

2.2 Correlation Analysis

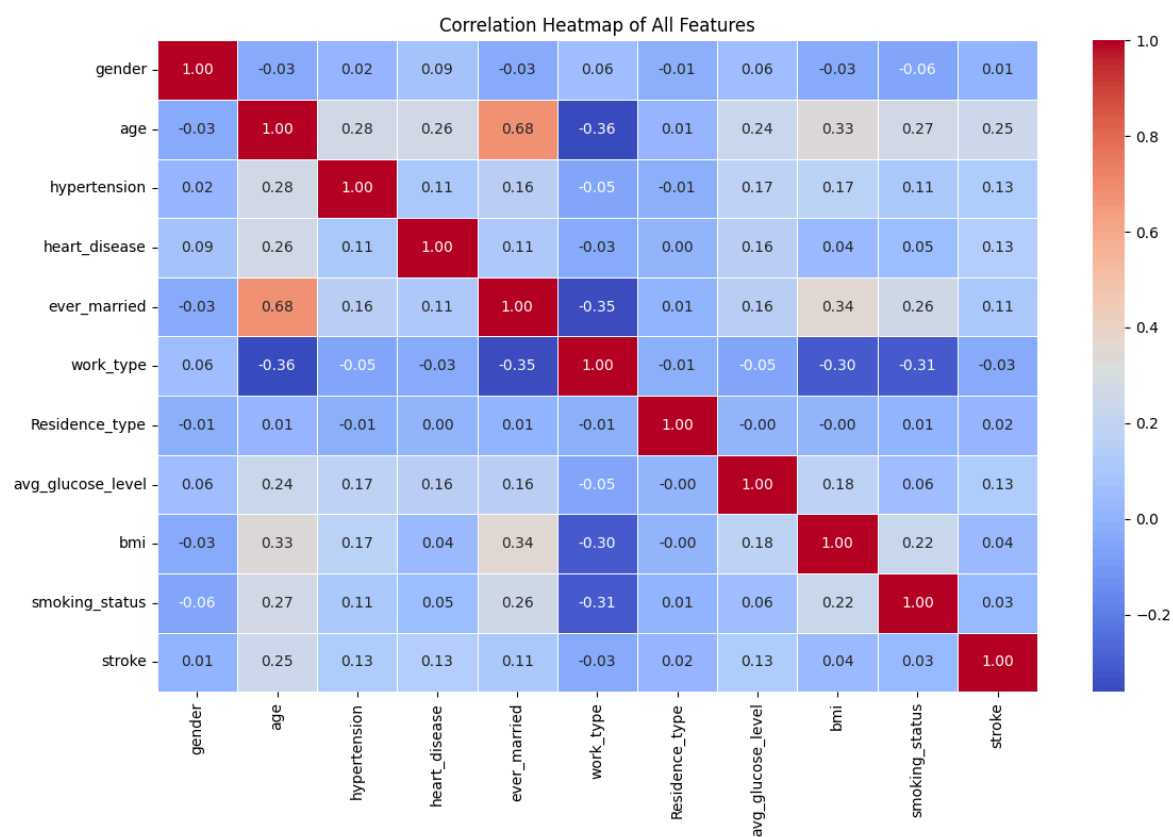


Figure 1: Correlation Heatmap of All Features

From the correlation heatmap, we observe:

- `age` has the highest positive correlation with `stroke`, indicating older patients are more likely to have strokes.
- `hypertension` and `heart_disease` show moderate positive correlation with `stroke`.
- `avg_glucose_level` has a weak positive correlation with `stroke`.

- Features like `Residence_type` show very weak correlation with `stroke`.
- Overall, correlations with `stroke` are relatively weak, making this a challenging classification task.

2.3 Imbalanced Dataset

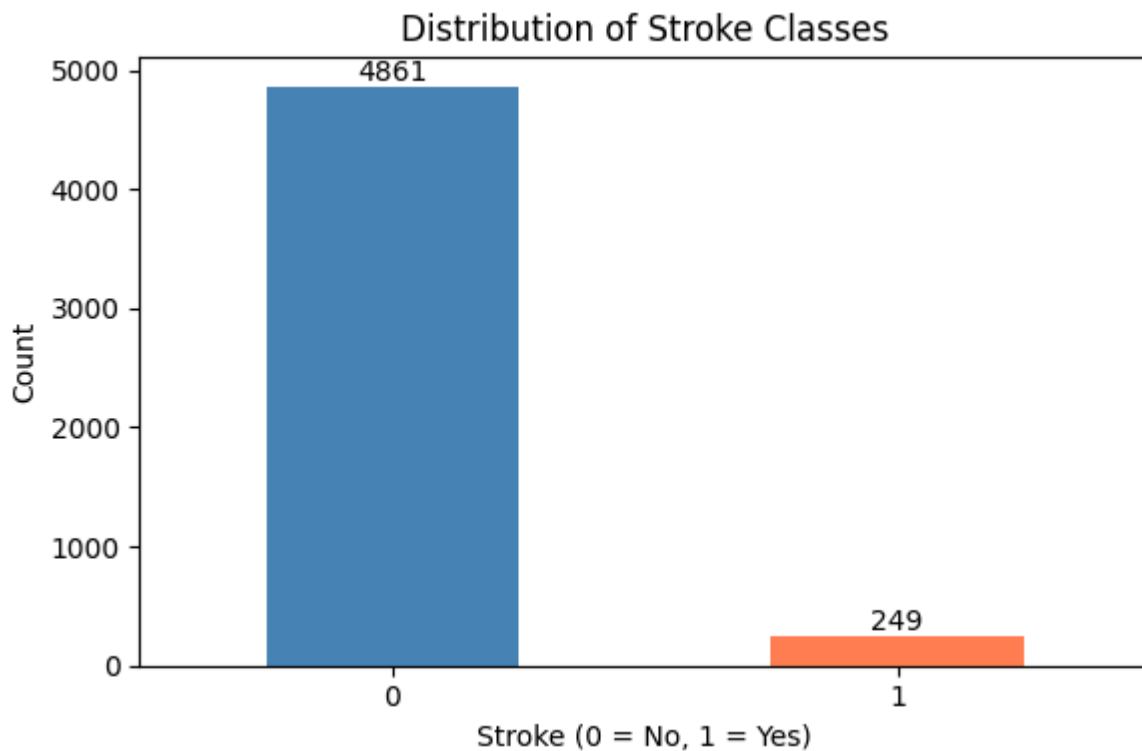


Figure 2: Distribution of Stroke Classes

Class	Count	Percentage
No Stroke (0)	~4,861	~95.13%
Stroke (1)	~249	~4.87%

The dataset is **heavily imbalanced**. The vast majority of patients did not have a stroke. This imbalance can cause models to be biased toward predicting the majority class, achieving high accuracy simply by always predicting "no stroke." We address this by applying **random oversampling** on the training set after splitting.

2.4 Exploratory Data Analysis

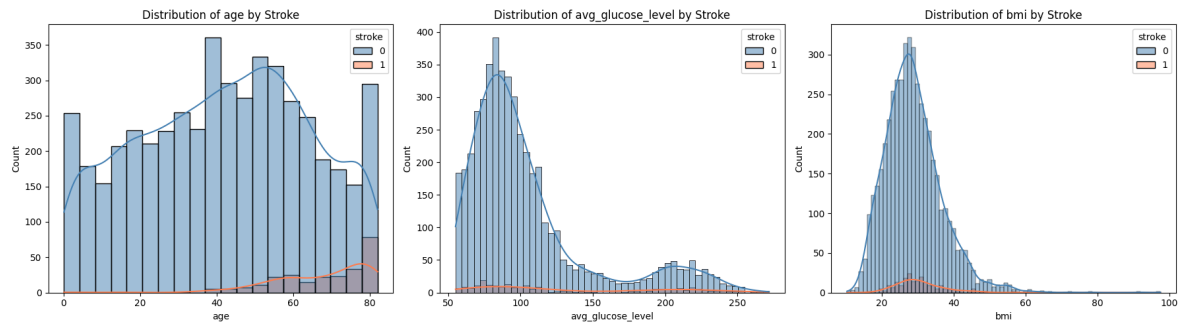


Figure 3: Distribution of Numerical Features by Stroke

Key EDA findings:

- Stroke patients tend to be significantly older. The age distribution for stroke cases is skewed toward higher ages.
- Higher `avg_glucose_level` is more common among stroke patients.
- Patients with hypertension or heart disease have noticeably higher stroke rates.
- Married individuals show higher stroke rates, likely because marriage correlates with older age.
- `Residence_type` (Urban vs. Rural) shows minimal difference in stroke rates.

3. Dataset Pre-processing

3.1 Problem: Irrelevant Feature (`id`)

The `id` column is a unique patient identifier with no predictive value for stroke prediction.

Solution: Drop the `id` column from the dataset.

3.2 Problem: Missing Values

The `bmi` column contains 201 missing (NaN) values out of 5,110 records.

Solution: Impute missing values with the **median** of the `bmi` column. We chose median over mean because median is robust to outliers (BMI has some extreme values). We chose imputation over deletion to preserve data points in an already small dataset, especially since the minority class (stroke) has very few samples.

3.3 Problem: Rare Category

The `gender` column contains an "Other" category with only 1 sample out of 5,110.

Solution: Remove this single row. It is too rare to be useful for training and can cause issues during stratified splitting.

3.4 Problem: Categorical Variables

Categorical features (`gender`, `ever_married`, `work_type`, `Residence_type`, `smoking_status`) are text-based and cannot be directly used by ML models.

Solution: Apply **One-Hot Encoding** using `pd.get_dummies()` with `drop_first=True` to convert categorical variables into binary columns while avoiding multicollinearity. After encoding, the dataset expands from 11 features to 16 features.

3.5 Feature Scaling

Numerical features have very different scales (e.g., `age`: 0–80, `avg_glucose_level`: 50–270, `bmi`: 10–97). Models like Neural Networks and Logistic Regression are sensitive to feature scale.

Solution: Apply **StandardScaler** (z-score normalization: subtract mean, divide by standard deviation). The scaler is fit *only on the training set* and then used to transform validation and test sets. This prevents data leakage.

4. Dataset Splitting

- **Method:** Stratified splitting — ensures the class distribution (95/5 ratio) is preserved in all subsets.
- **Train set:** 70%
- **Validation set:** 15%
- **Test set:** 15%

Handling class imbalance: After splitting, we apply **random oversampling** to the training set only. The minority class (stroke = 1) is duplicated with replacement until it matches the majority class count. This is done only on the training set to avoid data leakage into validation/test sets.

5. Model Training & Testing

5.1 Neural Network (MLPClassifier)

We use scikit-learn's `MLPClassifier` (Multi-Layer Perceptron), a feedforward neural network.

Hyperparameter	Value
Hidden layers	(64, 32) — two hidden layers with 64 and 32 neurons
Activation	ReLU
Solver	Adam
Max iterations	500

5.2 Logistic Regression

Logistic Regression is a simple linear model well-suited for binary classification. It models the probability of the positive class using the logistic (sigmoid) function.

Hyperparameter	Value
Max iterations	1000
Random state	42

5.3 Decision Tree

Decision Tree is a non-linear model that splits data based on feature thresholds. It is easy to interpret and understand.

Hyperparameter	Value
Max depth	5
Random state	42

5.4 KMeans Clustering (Unsupervised)

We treat the problem as an unsupervised learning task by applying **KMeans (k=2)** on the test features (without using labels). The goal is to see if KMeans can naturally discover clusters that correspond to stroke/no-stroke groups.

We use **PCA** (Principal Component Analysis) to reduce features to 2 dimensions for visualization, and compare KMeans cluster assignments to actual stroke labels.

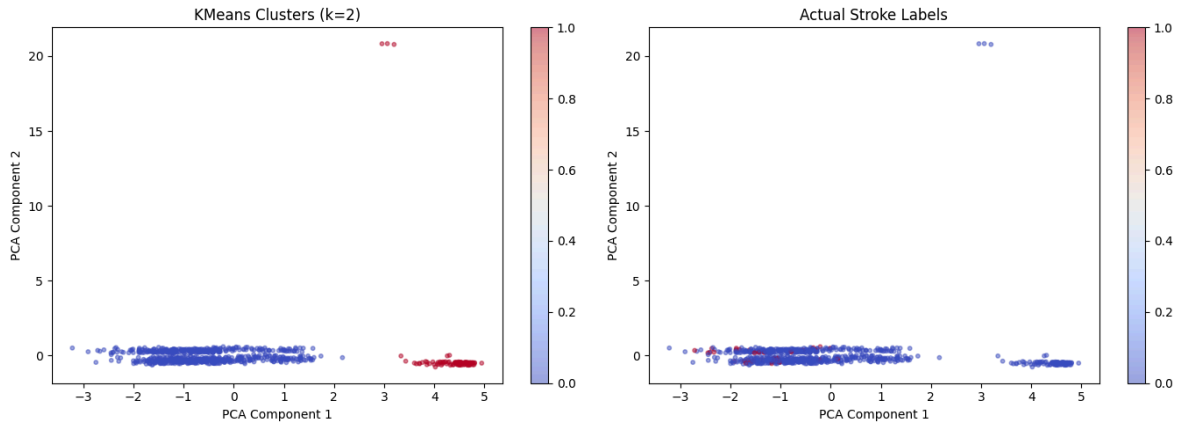


Figure 4: KMeans Clusters vs Actual Stroke Labels (PCA 2D)

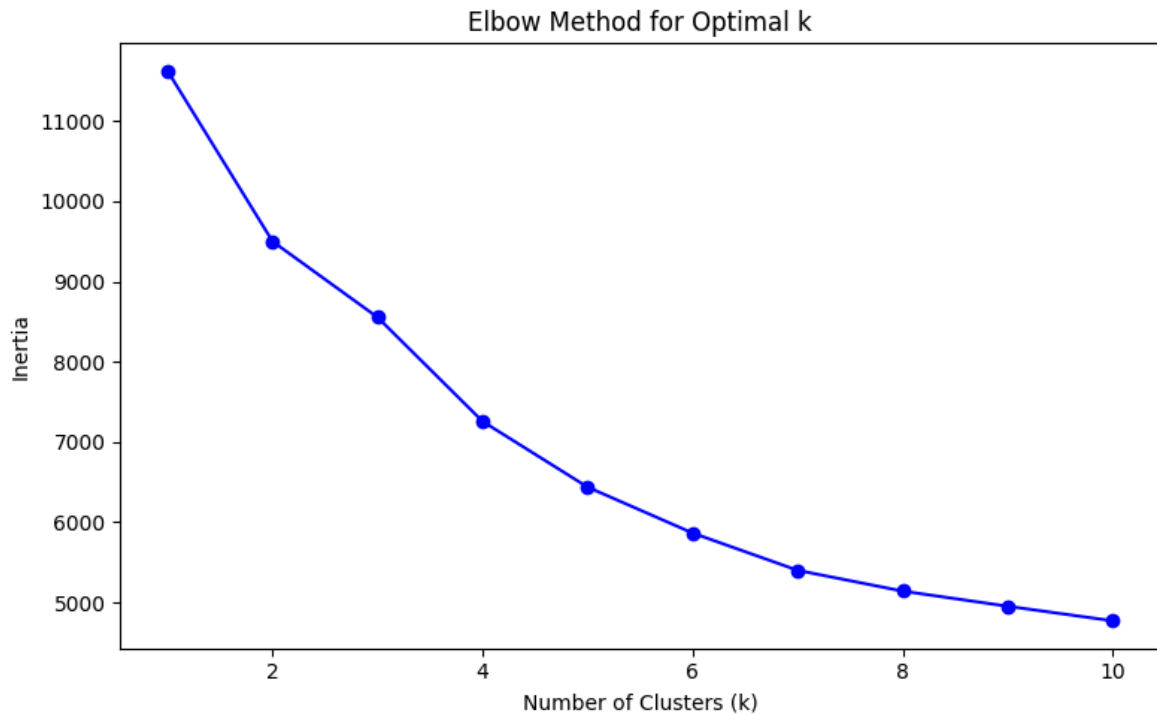


Figure 5: Elbow Method for Optimal k

The Elbow method plot validates k=2 as a reasonable choice. However, KMeans struggles to cleanly separate stroke vs. no-stroke classes due to weak feature separability.

6. Model Selection / Comparison Analysis

Comparison Table

Model	Accuracy	Precision	Recall	F1 Score	AUC
Neural Network	0.8983	0.1429	0.2105	0.1702	0.7385
Logistic Regression	0.7327	0.1354	0.8158	0.2322	0.8366
Decision Tree	0.7001	0.1160	0.7632	0.2014	0.7716

Accuracy Comparison

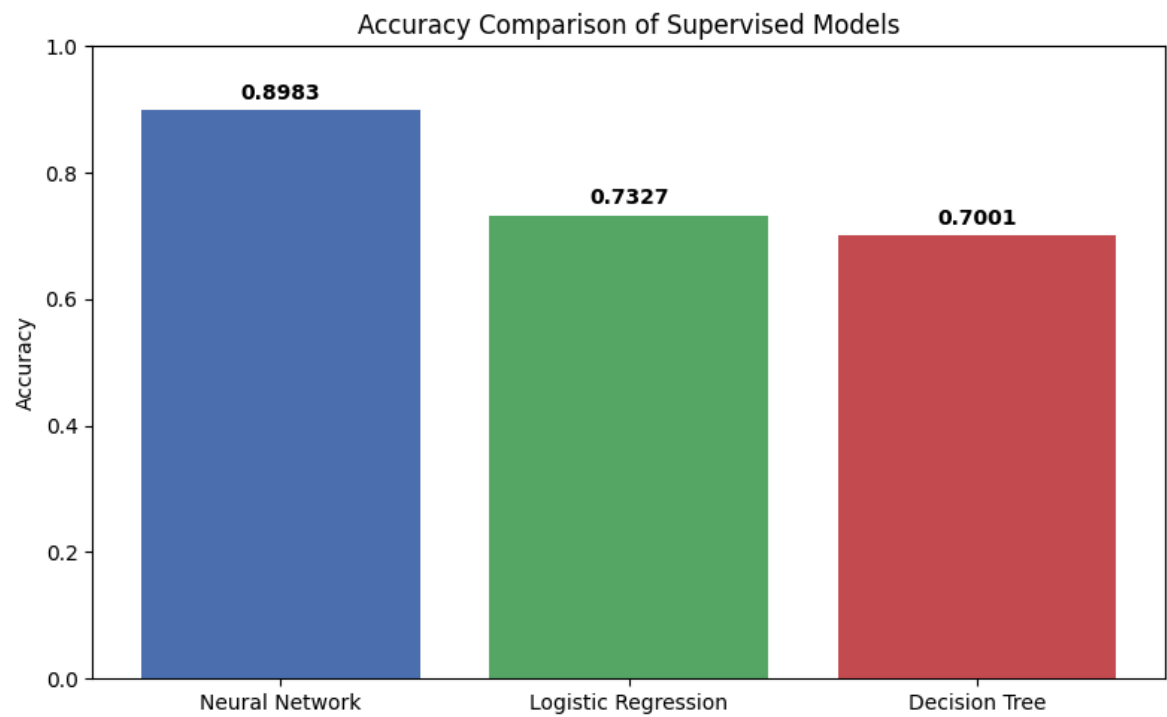


Figure 6: Accuracy Comparison of Supervised Models

Neural Network achieves the highest accuracy (89.83%), followed by Logistic Regression (73.27%) and Decision Tree (70.01%). However, accuracy alone is **misleading** for imbalanced datasets — a model that always predicts "no stroke" would achieve ~95% accuracy.

Precision & Recall Comparison

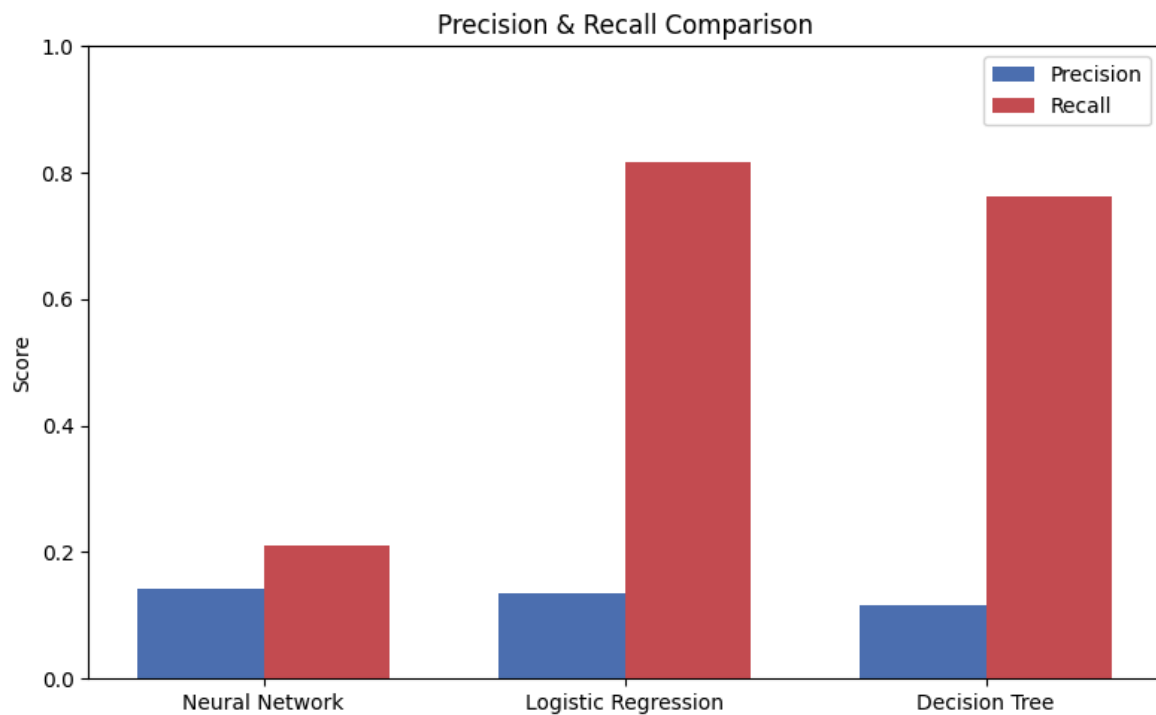


Figure 7: Precision & Recall Comparison

Logistic Regression achieves the best recall (81.58%), meaning it correctly identifies the most actual stroke cases. Decision Tree also has strong recall (76.32%). Neural Network has the worst recall (21.05%) — it misses most stroke cases despite having high accuracy.

Precision is low across all models (11–14%), indicating many false positives. This is a direct consequence of oversampling and the inherent difficulty of the problem.

Confusion Matrices



Figure 8: Confusion Matrices for All Models

The confusion matrices visually confirm the trade-off: Neural Network has few false positives but many false negatives (missed strokes). Logistic Regression and Decision Tree have more false positives but successfully catch more actual stroke cases.

ROC Curves & AUC Scores

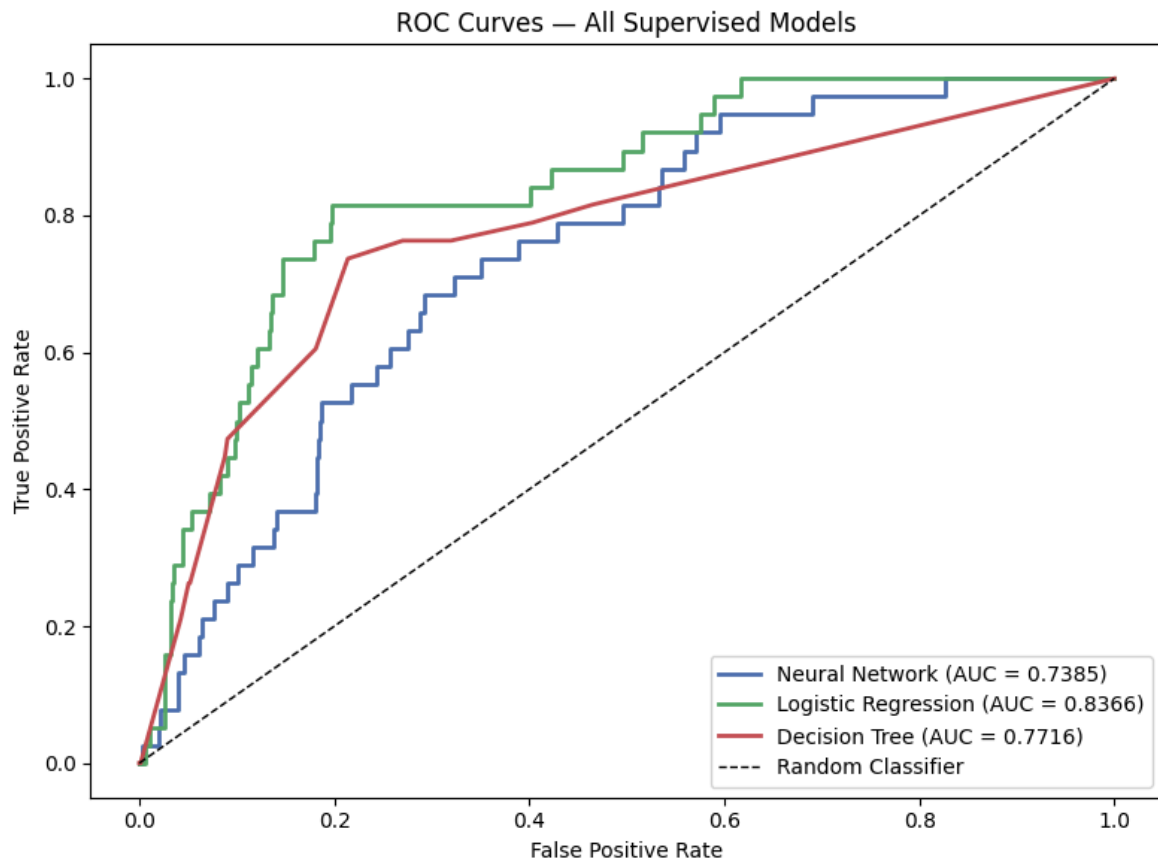


Figure 9: ROC Curves with AUC Scores

The ROC curve plots True Positive Rate vs. False Positive Rate. The AUC (Area Under the Curve) score is a more reliable metric than accuracy for imbalanced datasets. **Logistic Regression has the best AUC (0.8366)**, indicating it has the best overall discriminative ability between stroke and non-stroke cases.

7. Conclusion

What do we understand from the results?

Stroke prediction from patient health data is a challenging task due to severe class imbalance and weak feature correlations. No single feature is a strong predictor of stroke. Among the three supervised models, **Logistic Regression performed best overall** with the highest AUC score (0.8366) and recall (81.58%), despite having a lower raw accuracy than the Neural Network.

Model Performance Comments

- The Neural Network achieves the highest accuracy (89.83%) but this is deceptive — it primarily predicts the majority class ("no stroke") and misses most actual stroke cases (recall = 21%).
- Logistic Regression and Decision Tree, while having lower accuracy, are much better at identifying actual stroke patients, which is clinically more important.
- For healthcare applications, **recall is more important than precision** — it is better to flag a healthy patient for further testing (false positive) than to miss an actual stroke patient (false negative).
- The KMeans unsupervised approach struggles to separate stroke/no-stroke classes, confirming that the features do not form naturally separable clusters.

Why are we getting such results?

- The dataset is **extremely imbalanced** (~95% vs ~5%), which makes the classification task inherently difficult.
- The correlations between individual features and stroke are **weak**, meaning no feature alone is a strong predictor.
- The minority class has only ~249 samples, which limits the model's ability to learn complex patterns for stroke cases.
- Random oversampling helps but introduces duplicate samples, which can lead to overfitting on the training set.

Challenges Faced

1. **Severe class imbalance** — only ~5% of samples are positive (stroke), requiring oversampling techniques.
2. **Missing BMI values** — 201 records had missing BMI data, requiring imputation decisions.
3. **Weak feature correlations** — no single feature strongly predicts stroke, making the problem inherently difficult.
4. **Trade-off between precision and recall** — improving recall (catching more strokes) inevitably increases false positives (lower precision).

Possible Improvements

- Use more advanced oversampling techniques such as SMOTE (Synthetic Minority Over-sampling Technique).
- Try ensemble methods like Random Forest or Gradient Boosting, which often perform better on imbalanced data.
- Perform hyperparameter tuning using cross-validation to optimize each model.
- Collect more data, especially positive stroke cases, to improve minority class representation.
- Engineer new features (e.g., age groups, BMI categories) to capture non-linear relationships.